WIKIPÉDIA
L'encyclopédie libre

Mémoire virtuelle

En informatique, la **mémoire virtuelle** est une abstraction idéalisée des ressources de stockage réellement disponibles sur une machine donnée[1]. Il repose sur l'utilisation de traduction à la volée des adresses (virtuelles) vues du logiciel, en adresses physiques de mémoire vive ou de mémoire de masse. La mémoire virtuelle permet :

- d'utiliser de la mémoire de masse comme extension de la mémoire vive ;
- d'augmenter le taux de multiprogrammation ;
- de mettre en place des mécanismes de protection de la mémoire ;
- de partager la mémoire entre processus.

Ce mécanisme a été mis au point dans les années 1960.

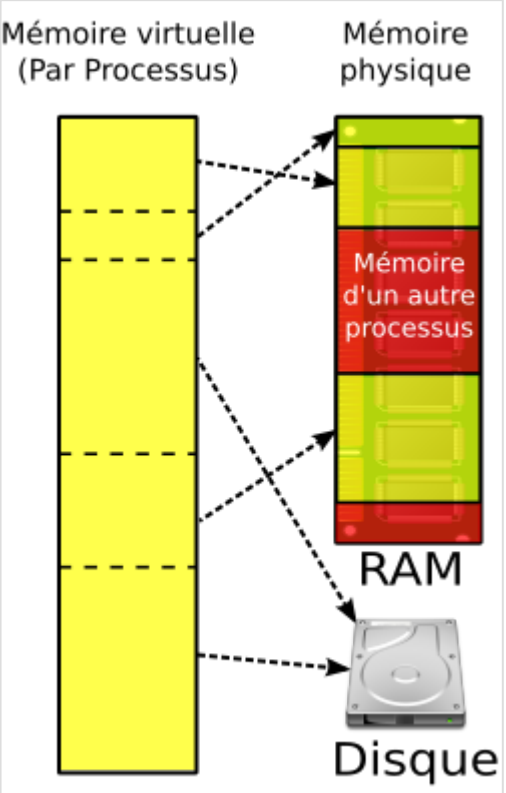


Schéma de principe de la mémoire virtuelle.

Historique

L'article de référence de James Kilburn, paru en 1962, décrit le premier ordinateur doté d'un système de gestion de mémoire virtuelle paginée et utilisant un tambour comme extension de la mémoire centrale à tores de ferrite : l'Atlas.

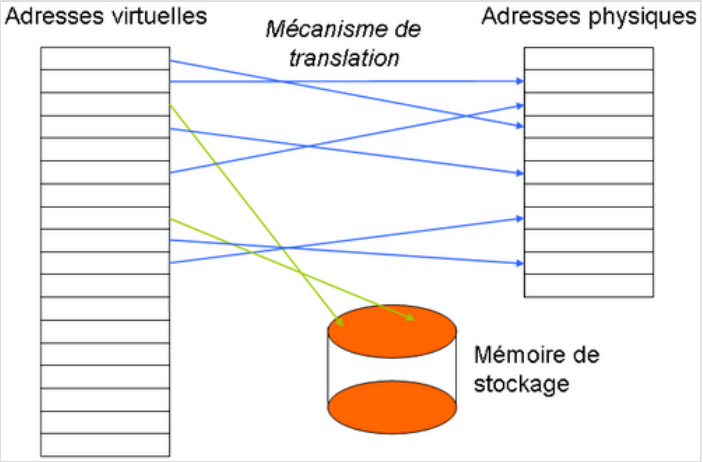
Aujourd'hui, tous les ordinateurs ont un mécanisme de gestion de la mémoire virtuelle, sauf certains supercalculateurs ou systèmes embarqués temps réel.

Mémoire virtuelle paginée

Le principe est le suivant :

- Les adresses mémoires émises par le processeur sont des adresses virtuelles, indiquant la position d'un mot dans la mémoire virtuelle.
- Cette mémoire virtuelle est formée de zones de même taille, appelées *pages*. Une adresse virtuelle est donc un couple (numéro de page, déplacement dans la page). La taille des pages est une puissance entière de deux, de façon à déterminer sans calcul le déplacement (10 bits de poids faible de l'adresse virtuelle pour des pages de 1 024 mots), et le numéro de page (les autres bits).
- La mémoire vive est également composée de zones de même taille, appelées cadres (*frames* en anglais), dans lesquelles prennent place les pages (un cadre contient une page : taille d'un cadre = taille d'une page). La taille de l'ensemble des cadres en mémoire vive utilisés par un processus est appelé *Resident set size*.

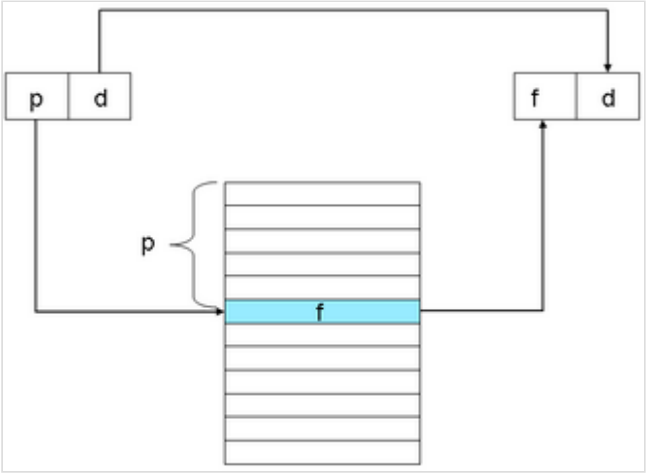
- Un mécanisme de traduction (*translation*, ou génération d'adresse) assure la conversion des adresses virtuelles en adresses physiques, en consultant une *table des pages* (*page table* en anglais) pour connaître le numéro du cadre qui contient la page recherchée. L'adresse physique obtenue est le couple (numéro de cadre, déplacement).
- Il peut y avoir plus de pages que de cadres (c'est là tout l'intérêt) : les pages qui ne sont pas en mémoire sont stockées sur un autre support (disque), elles seront ramenées dans un cadre quand on en aura besoin.



On traduit des adresses virtuelles en adresses physiques, et certaines informations peuvent être temporairement placées sur un support de stockage.

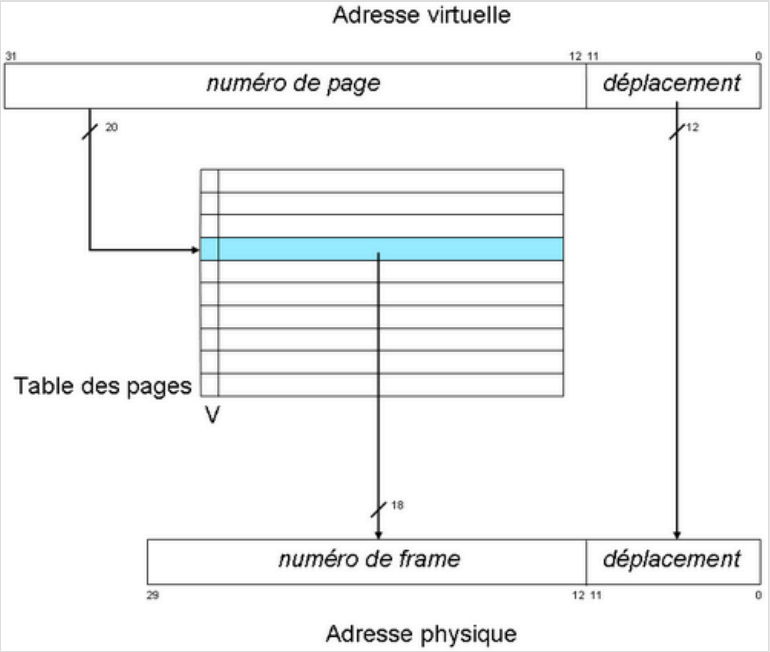
La table des pages est indexée par le numéro de page. Chaque ligne est appelée « entrée dans la table des pages » (*pages table entry*, abrégé PTE), et contient le numéro de cadre. La table des pages pouvant être située n'importe où en mémoire, un registre spécial (PTBR pour *Page Table Base Register*) conserve son adresse.

En pratique, le mécanisme de traduction fait partie d'un circuit électronique appelé MMU (*memory management unit*) qui contient également une partie de la table des pages, stockée dans une mémoire associative formée de registres rapides. Ceci évite d'avoir à consulter la table des pages (en mémoire) pour chaque accès mémoire.



Mémoire virtuelle : traduction du couple (page, déplacement), l'adresse virtuelle, en (frame, déplacement), l'adresse physique

Voici un exemple réel d'une machine dont le processeur génère des adresses virtuelles sur 32 bits, pouvant ainsi accéder à 4 Gio de mémoire. La taille de la page est de 4 Kio. On en déduit que le champ déplacement occupe les 12 bits de poids faible, et le champ numéro de page les 20 bits de poids fort.



Mémoire virtuelle : traduction du couple (page, déplacement), l'adresse virtuelle, en (frame, déplacement), l'adresse physique

On notera la présence d'un champ spécial appartenant à chaque PTE. Pour simplifier, nous avons réduit la largeur de ce champ à un bit : le bit de *validité*. Si celui-ci est à 0, cela signifie que le numéro de cadre est invalide. Il faut donc se doter d'une technique permettant de mettre à jour cette PTE pour la rendre valide.

Trois cas peuvent se produire :

1. L'entrée est valide : elle se substitue au numéro de page pour former l'adresse physique.
2. L'entrée dans la table des pages est invalide. Dans ce cas il faut trouver un cadre libre en mémoire vive et mettre son numéro dans cette entrée de la table des pages.
3. L'entrée dans la table des pages est valide mais correspond à une adresse sur la mémoire de masse où se trouve le contenu du cadre. Un mécanisme devra ramener ces données pour les placer en mémoire vive.

Allocation à la demande

Dans les deux derniers cas, une interruption – appelée *défaut de page* (ou parfois *faute de page*, traduction de l'anglais *page fault*) est générée par le matériel et donne la main au système d'exploitation. Celui-ci a la charge de trouver un cadre disponible en mémoire centrale afin de l'allouer au processus responsable de ce défaut de page, et éventuellement de recharger le contenu de cette *frame* par le contenu sauvé sur la mémoire de masse (couramment le disque dur, sur une zone appelée *zone d'échange* ou *swap*).

Il se peut qu'il n'y ait plus aucun cadre libre en mémoire centrale : celle-ci est alors occupée à 100 %. Dans ce cas un algorithme de pagination a la responsabilité de choisir une page « victime ». Cette page se verra soit immédiatement réaffectée au processus demandeur, soit elle sera d'abord sauvegardée sur

disque dur, et l'entrée de la table des pages qui la référence sera mise à jour. La page victime peut très bien appartenir au processus qui manque de place.

Ci-dessous sont listés quelques exemples d'algorithmes. La première ligne correspond à la *chaîne de références*, c'est-à-dire l'ordre dans lequel le processus va accéder aux pages. On suppose que la mémoire centrale est composée de trois *frames*. La *frame* victime apparaîtra soulignée. Les défauts de page initiaux ne sont pas comptés (ils sont en nombre identique quel que soit l'algorithme choisi).

- *L'algorithme optimal* : le nombre de défauts de page est réduit à 6. La règle de remplacement est « remplacer la *frame* qui ne sera pas utilisée pendant la durée la plus longue ». Malheureusement, cet algorithme nécessiterait de connaître l'avenir. Les autres algorithmes essayeront donc d'approcher cette solution optimale.

7	**0**	**1**	**2**	**0**	**3**	**0**	**4**	**2**	**3**	**0**	**3**	**2**	**1**	**2**	**0**	**1**	**7**	**0**	**1**
7	7	7	2		2		2			2				2				7	
	0	0	0		0		4			0				0				0	
		1	1		3		3			3				1				1	

- FIFO (*First in, first out* ou « Premier rentré, premier sorti ») : le cadre victime est celui qui a été amené en mémoire il y a le plus longtemps (le plus « ancien »). Notez qu'il n'est pas nécessaire de conserver l'instant auquel un cadre a été remplacé : il suffit de maintenir une structure FIFO, de remplacer le cadre dont le numéro apparaît en tête, et d'insérer le numéro du nouveau cadre en dernière position. Cet algorithme donne lieu à 12 remplacements :

7	**0**	**1**	**2**	**0**	**3**	**0**	**4**	**2**	**3**	**0**	**3**	**2**	**1**	**2**	**0**	**1**	**7**	**0**	**1**
7	7	7	2		2	2	4	4	4	0		0	0		7	7	7		
	0	0	0		3	3	3	2	2	2		1	1		1	0	0		
		1	1		1	0	0	0	3	3		3	2		2	2	1		

- L'algorithme le plus souvent utilisé est appelé LRU (*Least recently used*, soit « la moins récemment utilisée »). Il consiste à choisir comme victime le cadre qui n'a pas été référencé depuis le plus longtemps. On peut l'implémenter soit en rajoutant des bits dans chaque entrée de la table des pages qui indiquent quand a eu lieu la dernière référence à cette entrée, soit via une structure de liste où l'on amènera en première position le cadre récemment référencé, les cadres victimes restant donc en dernières positions. Cet algorithme donne lieu à 9 remplacements :

7	**0**	**1**	**2**	**0**	**3**	**0**	**4**	**2**	**3**	**0**	**3**	**2**	**1**	**2**	**0**	**1**	**7**	**0**	**1**
7	7	7	2		2		4	4	4	0			1		1		1		
	0	0	0		0		0	0	3	3			3		0		0		
		1	1		3		3	2	2	2			2		2		7		

- Autres algorithmes :
  - Remplacement aléatoire : où la *frame* victime est choisie au hasard.
  - LFU (*Least frequently used* soit « la moins souvent utilisée ») : on garde un compteur qui est incrémenté à chaque fois que le cadre est référencé, et la victime sera le cadre dont le compteur est le plus bas. Inconvénient : au démarrage du programme quelques pages peuvent être intensément utilisées, puis plus jamais par la suite. La valeur du compteur sera si élevée qu'ils ne seront remplacés que trop tardivement. Il faut aussi gérer le cas de dépassement de capacité du compteur…

Il peut être relativement facile de trouver des cas pathologiques qui rendent un algorithme inutilisable. Par exemple, pour l'algorithme LRU, il s'agirait d'un programme qui utilise 5 pages dans une boucle sur une machine qui ne compte que 4 cadres'. Il va d'abord utiliser les 4 premiers cadres séquentiellement (1, 2, 3, 4) puis un défaut de page va survenir et c'est la page 1, la plus anciennement chargée, qui sera la victime. Les pages utilisées sont maintenant (5, 2, 3, 4). Puisque le programme boucle, il a besoin de la page 1 (à la suite de la page 5). Cette fois-ci, la page victime est la page 2, remplacée par la 1 : (5, 1, 3, 4), puis (5, 1, 2, 4), (5, 1, 2, 3), etc. Un défaut de page est généré à chaque itération…

Anomalie de Belady

Intuitivement, augmenter le nombre de cadres de pages (c'est-à-dire agrandir la mémoire centrale) doit réduire le nombre de défauts de page.

L'anomalie de Belady (1970) est un contre-exemple qui montre que ce n'est pas absolument vrai avec l'algorithme FIFO, en effet le lecteur pourra vérifier par lui-même que la séquence de références (3, 2, 1, 0, 3, 2, 4, 3, 2, 1, 0, 4) conduit à

- 9 défauts de page avec 3 cadres,
- 10 défauts de page avec 4 cadres.

Remarque : il ne faut pas exagérer la portée de cette curiosité. Elle montre certes que l'algorithme FIFO n'a pas *en général* une propriété à laquelle on se serait attendu (ajouter de la mémoire réduit les défauts de page) mais elle ne montre pas qu'elle ne l'a pas *en moyenne*. Et de toute façon l'algorithme FIFO n'est jamais utilisé pour le remplacement de page.

Par ailleurs, on peut démontrer que certains algorithmes de remplacement de pages (LRU par exemple) ne sont pas sujets à ce type d'anomalie.

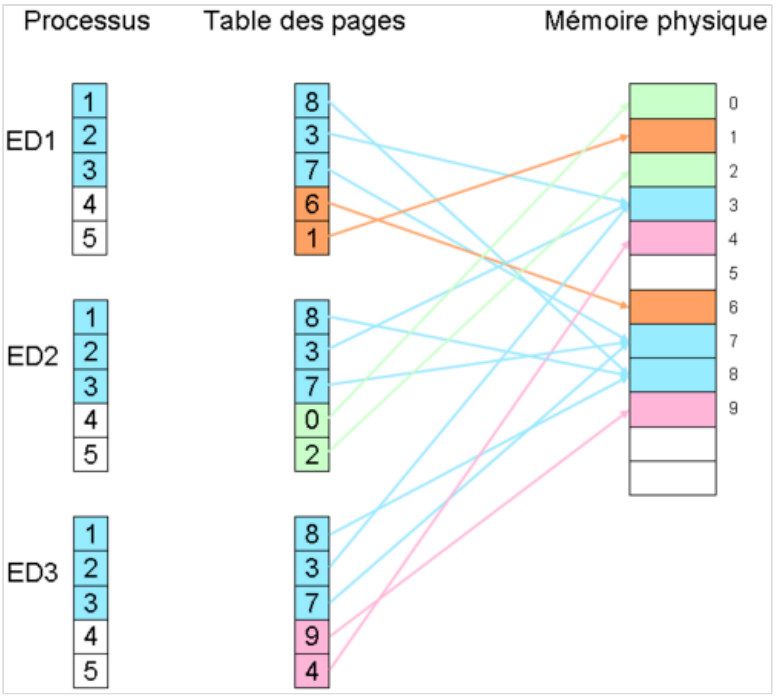
Méthode d'allocation dans un système multiprogrammé

Les méthodes de sélection de la page victime évoquées ci-dessus peuvent s'appliquer soit aux pages appartenant à un processus (on parle alors d'« allocation locale »), soit à toutes les pages et donc à toute la mémoire (dans ce cas la technique d'allocation est dite « globale »).

Dans un système d'allocation globale, le temps d'exécution d'un processus peut grandement varier d'une instance à l'autre car le nombre de défauts de page ne dépend pas du processus lui-même. D'un autre côté, ce système permet au nombre de cadres alloués à un processus d'évoluer.

Partage de mémoire dans un système paginé

Le schéma suivant montre trois processus en cours d'exécution, par exemple un éditeur de texte nommé Ed. Les trois instances sont toutes situées aux mêmes adresses virtuelles (1, 2, 3, 4, 5). Ce programme utilise deux zones mémoire distinctes : les pages qui contiennent le code, c’est-à-dire les instructions décrivant le programme, et la zone de données, le fichier en cours d'édition. Il suffit de garder les mêmes entrées dans la table des pages pour que les trois instances se partagent la zone de code. Par contre, les entrées correspondantes aux pages de données sont, elles, distinctes.



Mémoire virtuelle : partage du code entre trois processus constitués de 3 pages de code et 2 pages de données.

Protection

Des bits de protections sont ajoutés à chaque entrée de la table des pages. Ainsi on pourra aisément faire la distinction entre les pages allouées au noyau, en lecture seule, etc. Voir l'exemple ci-dessous.

Efficacité

On rencontre trois problèmes majeurs :

1. La taille de la table des pages : pour une architecture où 20 bits sont réservés pour le numéro de page, la table occupera 4 Mio de mémoire minimum (2^{20} = 1 Mio de PTE, chaque PTE ayant une longueur de 4 octets). Ce problème est résolu par l'utilisation de plusieurs tables de pages : le champ numéro de page sera décomposé en plusieurs, chacun indiquant un déplacement dans la table de plus bas niveau. Les VAX et les Pentiums supportent deux niveaux, le SPARC trois, les Motorola 680x0 quatre… On peut aussi segmenter la table des pages.
2. Le temps d'accès : la table des pages étant située en mémoire, il faudrait deux accès mémoire par demande de la part du processeur. Pour pallier ce problème les entrées les plus souvent utilisées sont conservées dans une mémoire associative (mémoire cache) appelée TLB pour *Translation Lookaside Buffer*. Chaque adresse virtuelle issue du processeur est cherchée dans le TLB ; s'il y a correspondance, l'entrée du TLB est utilisée, sinon une interruption est déclenchée et le TLB devra être remis à jour par l'entrée de la table des pages stockée en mémoire avant que l'instruction fautive ne soit redémarrée. Tous les microprocesseurs actuels possèdent un TLB.
3. Phénomène de *thrashing* (effondrement) : plus le taux de multiprogrammation augmente, moins chaque processus se voit allouer de pages. Au bout d'un moment, le système sature car trop de défauts de page sont générés. Le phénomène de *trashing* apparait à chaque fois que, dans un système de stockage hiérarchique, un des niveaux se voit surchargé. C'est par exemple le cas si la mémoire cache est trop petite. À ce moment-là les allers-retours incessants de données le long de la hiérarchie vont fortement diminuer le

rendement de l'ordinateur. Il est possible de diminuer les effets de ce comportement en rajoutant des ressources matérielles (ajouter de la mémoire), en diminuant le taux de multiprogrammation, ou en modifiant la priorité des processus.

Principe de localité

Le comportement des programmes n'est pas chaotique : le programme démarre, il fait appel à des fonctions (ou parties de code) qui en appellent d'autres à leur tour, etc. Chacun de ces appels définit une région. Il est probable que le programme puisse passer beaucoup de temps à s'exécuter au sein de quelques régions : c'est le principe de localité. Le même principe peut être appliqué aux pages contenant des données.

Autrement dit, un programme accède fréquemment à un petit ensemble de pages, et cet ensemble de pages évolue lentement avec le temps.

Si l'on est capable de conserver en mémoire ces espaces souvent accédés, on diminue les chances de voir un programme se mettre à *trasher*, c'est-à-dire réclamer des pages qu'on vient de lui retirer récemment.

Le *working set* : espace de travail

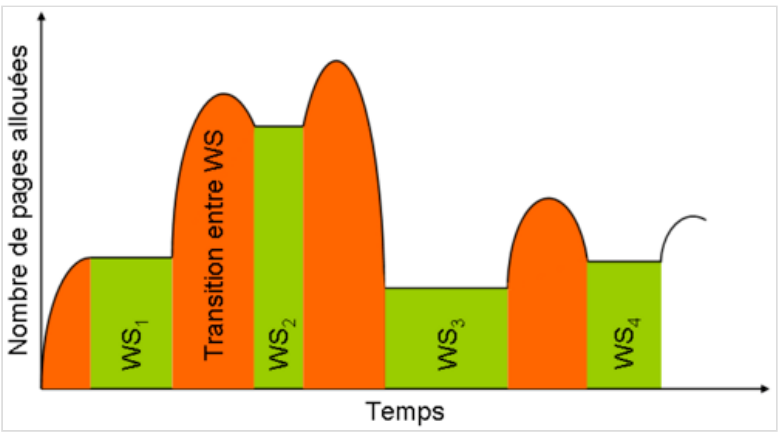
On peut définir un paramètre, Δ, qui est le nombre de références aux pages accédées par le processus durant un certain laps de temps. La figure ci-dessous montre la valeur de l'espace de travail à deux instants différents :



Visualisation du Working Set pour Δ = 10, aux instants

Il faut choisir la valeur de Δ avec soin : trop petite elle ne couvre pas l'espace de travail *nominal* du processus ; trop grande elle inclut des pages inutiles. Si Δ est égal à l'infini, il couvre la totalité du programme, bien sûr.

Pour un unique processus, on peut représenter graphiquement comment la mémoire lui est allouée, et visualiser les espaces de travail :



Les régions en orange représentent les transitions entre deux espaces de travail (en vert).

Les « plateaux » sont des zones où il n'y a pas de défaut de page : l'espace alloué est suffisamment grand pour contenir toutes les *frames* dont le processus a besoin pendant un temps relativement long. Les défauts de pages ont lieu dans la partie ascendante de la transition, tandis que de la mémoire est *libérée* quand la courbe retombe vers le prochain espace de travail : zone d'équilibre.

C'est au système d'exploitation de mettre en œuvre les algorithmes pour que la valeur de Δ soit optimum de sorte que le taux de multiprogrammation et l'utilisation de l'unité centrale soient maximisés. En d'autres termes : éviter le *trashing*. Si la somme des espaces de travail de chacun des processus est supérieur au nombre de *frames* disponibles, il y aura forcément effondrement.

Prépagination

Un des avantages de la mémoire virtuelle est de pouvoir commencer l'exécution d'un programme dès que sa première page de code est chargée en mémoire. La prépagination va non seulement charger la première page, mais les quelques suivantes, dont la probabilité d'être accédée est très élevée.

Taille des pages pour quelques ordinateurs

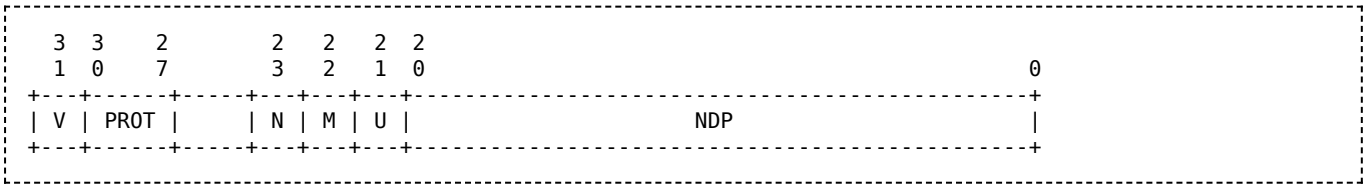
Voici indiqué en bits, l'espace total adressable, la largeur des champs numéro de page, et déplacement.

Machine	Espace adressable	Champs *numéro de page*	Champs "déplacement"
Atlas	2^{20}	11	9
PDP-10	2^{18}	9	9
IBM-370	2^{24}	13 ou 12	11 ou 12
Pentium Pro	2^{32}	12 ou 20	20 ou 12
Alpha 21064	2^{43}	13	30

Exemple

- Voici un exemple, tiré du manuel du Tahoe, un clone du VAX :

  Les adresses sont codées sur 32 bits (4 Gio d'espace total)
  La taille de la page est de 1 Kio (codée sur 10 bits).
  Les entrées dans la table des pages sont à ce format :



Les champs M, U, N et NDP ne sont valides que si le bit V est à 1. Quand V est à 0, le champ NDP contient l'adresse sur le disque dur où se trouve la page.

Le champ PROT doit être interprété comme ceci (la valeur du champ est donnée en binaire sur 4 bits) :

Valeur	Protection
0000	Aucun accès
1000	Lecture pour le noyau
1100	Lecture/écriture pour le noyau
1010	Lecture utilisateur et noyau
1110	Lecture utilisateur, lecture/écriture pour le noyau
1111	Lecture/écriture utilisateur et noyau

Le bit 24, *N* (Non-cachée), signifie que la page n'est pas en cache et que le système doit lire ou écrire directement depuis ou vers la mémoire.

Le bit *M* (Modifiée) est modifié par le matériel si le contenu de la page est modifié.

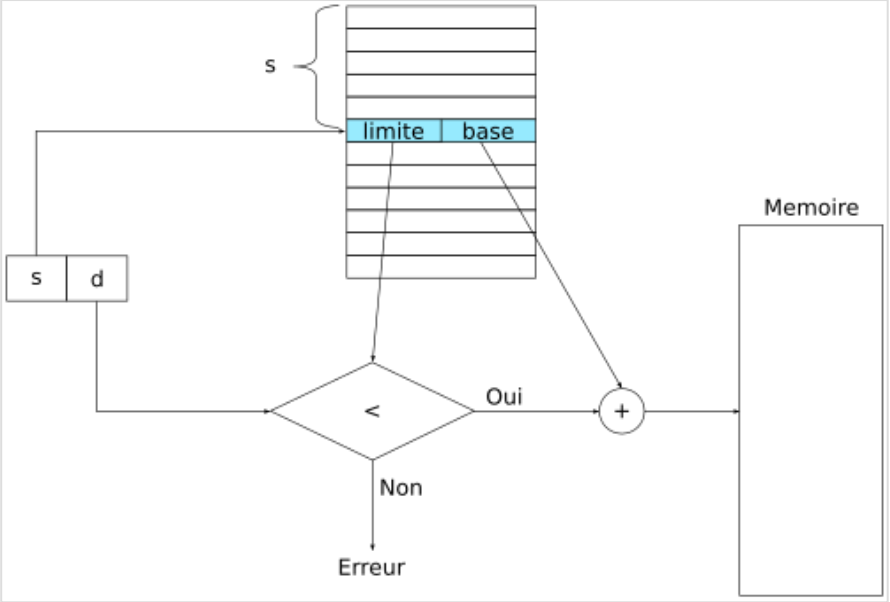
Le bit *U* (Utilisée) indique si la page a été lue ou écrite par un processus. Il est utile, en association avec les autres, pour la détermination de l'espace de travail (*Working Set*) d'un processus (cf. ci-dessus).

- L'appel système *vfork(2)* du système d'exploitation Unix crée un nouveau contexte (processus) en dupliquant la table des pages du processus qui fait l'appel (son *père*). La partie de la table des pages marquée en lecture seule (le code) sera dupliquée telle quelle. Les pages qui correspondent aux données sont marquées *copy on write*. Quand Unix devra effectuer une écriture sur une page marquée *copy on write*, il allouera une nouvelle *frame*, recopiera le contenu de la *frame* originale et enfin fera la modification demandée sur cette nouvelle *frame*. Finalement *vfork(2)* est donc un appel système peu coûteux car il ne fait pas grand-chose…
- Pour ceux qui savent lire les sources C d'Unix, la définition des PTE est donnée dans le fichier <…/pte.h> de diverses architectures. Un excellent exemple de la manière d'utiliser les PTE depuis un programme utilisateur est fourni dans le source du programme *ps* de BSD 4.3.

Segmentation

La segmentation offre une vue de la mémoire plus cohérente avec celle de l'utilisateur. En effet, celui-ci ne considère pas (ou rarement !) la mémoire comme une suite de pages mais plutôt comme des espaces, ou des régions, destinés à une utilisation particulière par exemple : le code d'un programme, les données, la pile, un ensemble de sous-programmes, des modules, un tableau, etc. La segmentation reflète cette organisation.

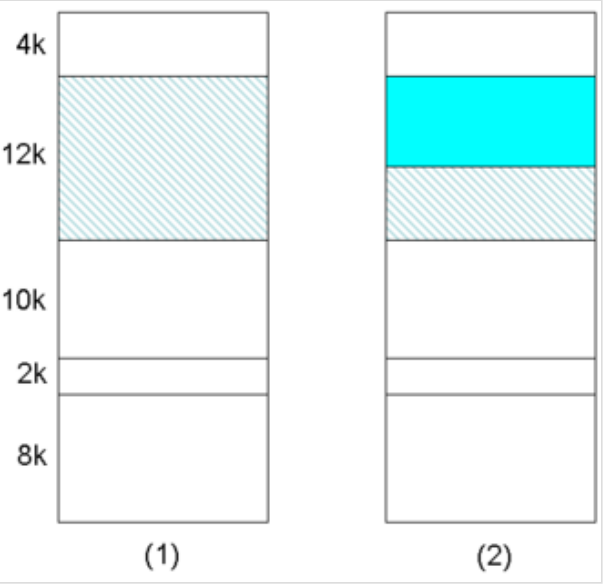
Chaque objet logique sera désigné par un segment. Dans un segment l'adressage se fera à l'aide d'un déplacement. Le couple (segment, déplacement) sera traduit en adresse mémoire par le biais d'une table de segments contenant deux champs, *limite* et *base*. La base est l'adresse de début du segment, et limite la dernière adresse du même segment :



Segmentation : l'adresse virtuelle issue du processeur à la forme (segment, déplacement). Elle est traduite en adresse physique par le biais d'une table de segments. Un test est effectué pour vérifier que l'adresse est bien dans l'intervalle du segment.

Problème de fragmentation

Les systèmes paginés rencontrent un problème de fragmentation *interne* : de la place est perdue à la fin d'une page. Les systèmes segmentés connaissent un problème de fragmentation *externe* : des espaces entre des segments sont trop petits pour loger de nouveaux fragments, cet espace est donc perdu.

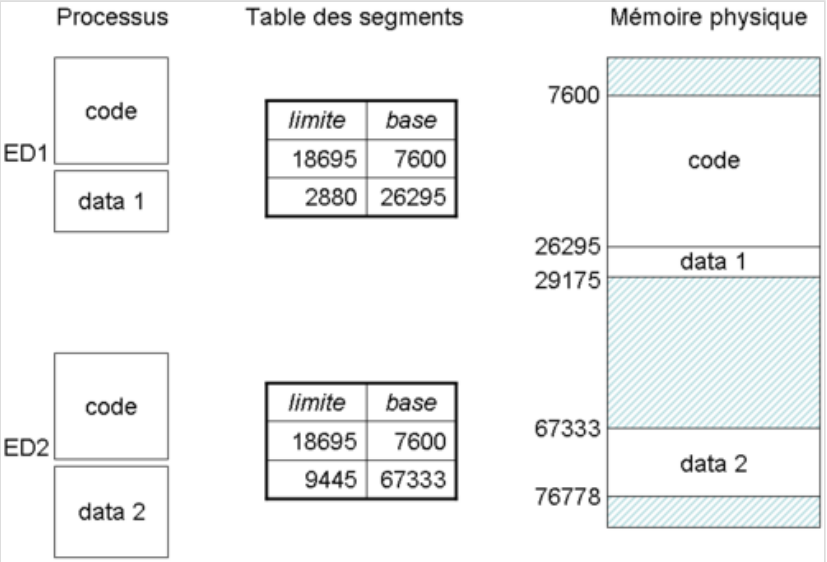


L'espace libre de 12 Kio de la disposition mémoire (1) diminue quand une partie est allouée à un nouveau segment. Néanmoins il n'est pas certain que le plus petit segment résiduel, visible en (2), sera assez grand pour répondre à la prochaine requête du système d'exploitation.

Il est possible de le récupérer en compactant la mémoire, c'est-à-dire en déplaçant les segments — tout en reflétant ces modifications dans les tables des segments — de sorte qu'ils soient contigus. Néanmoins cette opération est coûteuse.

Partage de segments

Il est possible de partager des segments entre processus, comme illustré sur la figure ci-dessous, où deux processus Ed1 et Ed2 partagent le même segment de code (programme) mais ont des segments pour les données disjoints et de tailles différentes.



Deux utilisateurs utilisant le même programme (un éditeur par exemple) se verront partager le même segment de code, mais pas de données.

Protection dans un système segmenté

Cette protection sera assurée par des bits supplémentaires ajoutés dans la table des segments, de la même façon que pour un système paginé.

Exemple de microprocesseurs à architecture mémoire segmentée

L'exemple le plus connu est l'Intel 8086 et ses quatre registres :

- CS, pour Code Segment : pointe vers le segment contenant le programme courant.
- DS, pour Data Segment : pointe vers le segment contenant les données du programme en cours d'exécution.
- ES, pour Extra Segment : pointe vers le segment dont l'utilisation est laissée au programmeur.
- SS, pour Stack Segment : pointe vers le segment contenant la pile.

Les successeurs du 8086 sont aussi segmentés :

- le 80286 peut gérer 16 Mio de mémoire vive et 1 Gio de mémoire virtuelle soit 16 384 segments de 64 Kio.

- le 80386 4 Gio de mémoire vive, 64 Tio de mémoire virtuelle, soit 16 384 segments de 4 Gio.

Systèmes mixtes paginés/segmentés

Il est possible de mixer les deux modes précédents :

1. la pagination segmentée, où la table des pages sera segmentée. Autrement dit, le numéro de page *p* du couple (p, d) de l'adresse virtuelle sera interprété comme un segment (s, p'). Ce système résout le problème de taille de la table des pages.
2. la segmentation paginée, où chaque segment sera paginé. Autrement dit, le champ déplacement *d* du couple (s, d) de l'adresse virtuelle sera interprété comme un numéro de page et un déplacement (p, d').

Swapping

Il est parfois nécessaire de supprimer *toutes* les pages ou segments d'un processus de la mémoire centrale. Dans ce cas le processus sera dit *swappé*, et toutes les données lui appartenant seront stockées en mémoire de masse. Cela peut survenir pour des processus *dormant* depuis longtemps, alors que le système d'exploitation a besoin d'allouer de la mémoire aux processus actifs. Les pages ou segments de code (programme) ne seront jamais *swappés*, mais tout simplement réassignés, car on peut les retrouver dans le fichier correspondant au programme (le fichier de *l'exécutable*). Pour cette raison, le système d'exploitation interdit l'accès en écriture à un fichier exécutable en cours d'utilisation ; symétriquement, il est impossible de lancer l'exécution d'un fichier tant qu'il est tenu ouvert pour un accès en écriture par un autre processus.

Compression de mémoire virtuelle

La compression de mémoire virtuelle permet d'améliorer la performance d'un système de mémoire virtuelle. Cette technique de gestion de mémoire virtuelle utilise la compression de données pour réduire la taille ou le nombre de requêtes de pagination vers et depuis le stockage auxiliaire.

Dans un système de compression de mémoire virtuelle, les pages sont compressées et stockées dans la mémoire physique, généralement une mémoire RAM, ou envoyées compressées vers un stockage auxiliaire, tel qu'un disque dur ou un disque SSD. Dans les deux cas, la plage de mémoire virtuelle dont le contenu a été compressé est inaccessible, de sorte que les tentatives d'accès aux pages compressées déclenchent des erreurs de page et inversent le processus de compression (récupération du stockage auxiliaire et décompression). L'empreinte des données paginées est réduite par le processus de compression et la mémoire RAM libérée est renvoyée dans le pool de mémoire physique disponible. Dans le cas où les pages compressées sont conservées dans la mémoire RAM, les pages compressées utilisent évidemment moins d'espace que les pages originales. Dans le cas où les pages compressées sont

conservées dans un stockage auxiliaire, la mémoire RAM est complètement libérée et les opérations d'écriture et de lecture sur la mémoire auxiliaire sont plus rapides que si les pages n'avaient pas été compressées[2].

Références

1. (en) Abhishek Bhattacharjee et Daniel Lustig, *Architectural and operating system support for virtual memory*, Morgan & Claypool, 2017 (ISBN 978-1-62705-602-1), p. 1
2. US Patent 5559978

Voir aussi

Articles connexes

- Mémoire paginée
- Page Attribute Table
- RAM Disque, qui est le principe inverse de la mémoire virtuelle (donc, utiliser la RAM comme une mémoire de masse).
- Unité de gestion mémoire

Bibliographie

- *One level storage system*, Kilburn, Edwards, Lanigan, Summer, IRE Transactions on elecronic computers, EC-11, vol. 2, avril 1962, p. 223-235.
- *Computer Organization and Design*, Hennessy, Patterson, Morgan Koffman, (ISBN 1558604286).
- *Operating system concepts*, Patterson, Silberschatz, (ISBN 020151379X).
- *Computer Organisation & Architecture*, Hennessy, Patterson, Morgan Koffman, (ISBN 0333645510).
- *Computer Architecture : A Quantitative Approach*
- *Computation Structures*, Stephen A. Ward, Robert H. Halstead, (ISBN 026273088X).
- *Structured Computer Organisation*
- *VAX reference Manual*
- *Sperry 7000/40 Architecture and Assembly Language Manual*

Ce document provient de « https://fr.wikipedia.org/w/index.php?title=Mémoire_virtuelle&oldid=239664267 ».